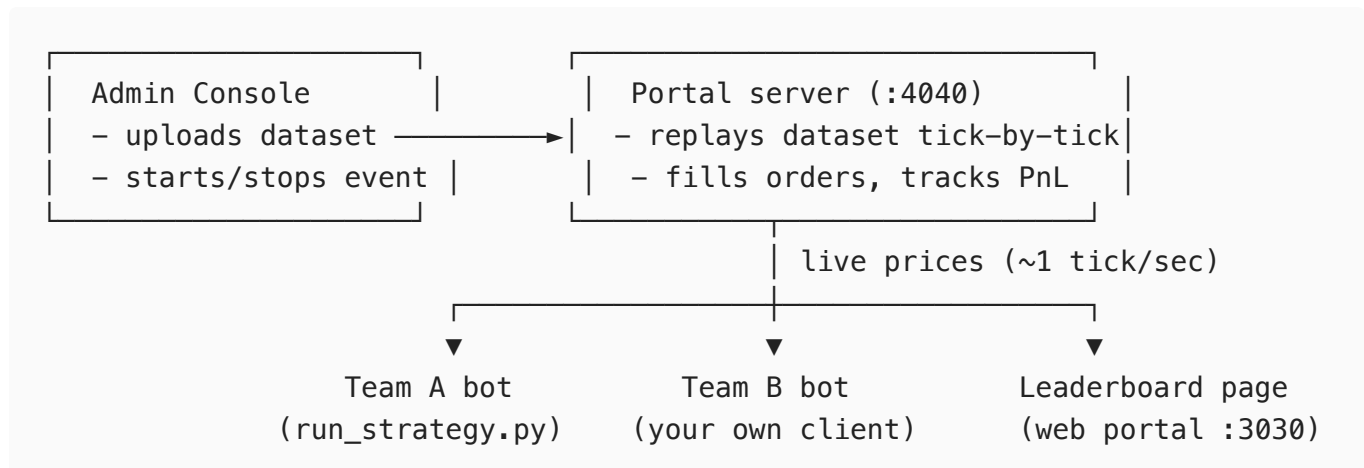


User Guide

Mercatus Arena : How to Run Your Algorithm

0. How the system fits together



You write a **strategy**, the provided **runner** executes it against the live feed, and every order goes to the server which is the single source of truth.

You never need the raw dataset — prices stream to you in real time.

1. Prerequisites

Tool	Version	Check
Python	3.10+	<code>python3 --version</code>
Node.js	20+	<code>node -v</code>
Docker	any recent	<code>docker info</code>

The runner uses **Python stdlib only** — no `pip install` required.

2. Get an account

Option A — self-register (when registration is open)

```
curl -X POST http://localhost:4040/api/auth/register \
-H 'Content-Type: application/json' \
-d '{"team_name":"Team
Rocket","email":"rocket@uni.edu","password":"StrongPass#1"}'
```

You get back a JWT and a **masked API key** (`sk_abc...xyz`). Organizers reveal full keys before launch.

3. Create your config file

Copy the template and fill in your credentials:

```
{
  "server": {
    "base_url": "--URL--",
    "ws_url": "ws://--URL--/ws"
  },
  "auth": {
    "api_key": "sk_your_full_key",
    "email": "rocket@uni.edu",
    "password": "StrongPass#1"
  },
  "schedule": { "from": "00:00:00", "to": "23:59:59" }
}
```

- `base_url` — ask organizers for the event URL (`http://localhost:4040` when running locally).
 - `api_key` — used for trading orders. If keys aren't revealed yet, leave `""`; the runner logs in with email/password instead.
 - Keep this file out of git — it contains secrets.
-

4. Run the algorithm

Quick start (provided strategy)

```
cd mercatus-arena
python3 run_strategy.py my_config.json
```

That's it. You should see:

```
connecting to http://localhost:4040
strategy.py online | symbols: ['AAPL', 'MSFT', 'GOOG', 'TSLA', 'NVDA']
[tick ] eq≈10,000,000 pos=[]
[fill] BUY AAPL x118 @ 109.31 cashAfter=9,870,120
```

Useful flags/environment:

What	How
Different config	<code>python3 run_strategy.py other_config.json</code>
Faster/slower ticks	<code>RUN_POLL=2.5 python3 run_strategy.py ...</code> (seconds between polls)
Run in background	<code>nohup python3 -u run_strategy.py my_config.json > run.log 2>&1 &</code>
Stop	<code>kill <PID></code> (PID printed at launch)

Tip: always use `python3 -u` when redirecting output, otherwise logs are buffered and appear empty.

5. Write your own strategy

`run_strategy.py` imports `strategy.py` from the repo root. Your module must expose exactly one function:

```
def strategy(current_data, portfolio, cash, history):
    """
    current_data : {symbol: {"high": p, "low": p, "close": p, "volume": v}}
    portfolio    : {symbol: {"quantity": int}} # what you currently hold
    cash         : float                       # current cash balance
    history       : list of past price dicts    # grows each tick

    return: {}                                -> no action
           {sym: ("BUY", qty)}                 -> market order
           {sym: ("SELL", qty)}
           "HOLD"                              -> no action
    """
```

Rules of thumb:

1. **Warm up** — collect ≥30 ticks before trading (indicators need history).

2. **Trust the server** — re-read positions/cash every tick (the runner does this for you); never cache quantities across ticks.
3. **Handle rejects** — one bad order should never crash the loop.
4. **Size by risk** — cap exposure, keep cash reserve, set stop-losses.
5. **Respect limits** — max **60 trades/min**, **300 API calls/min**.

Test offline before the event with the simulator harness:

```
python3 test_algo.py          # runs strategy.py against historical data
```

6. Where the data comes from (dataset lifecycle)

Students do not upload or download datasets. Organizers load one before the event; the server replays it tick-by-tick with slight random noise so every event differs.

Format

CSV with columns `t,symbol,price,volume`:

```
t,symbol,price,volume
1000,AAPL,109.3120,542
1000,MSFT,182.5254,318
2000,AAPL,109.3387,611
...
```

- `t` — milliseconds since replay start (rows sorted ascending)
- One row per symbol per tick; spacing typically 1000 ms
- Prices rounded to 4 decimals

During the event

- New prices publish ~every second; poll `GET /api/market/snapshot` every 0.5–1 s, or subscribe to the WebSocket feed (`ws://<host>/ws`) for pushes.
 - Admins may inject **flash crashes** mid-event — always use stops.
 - Check `GET /api/market/status` for state (`ACTIVE_MARKET` = you can trade).
-

7. Monitor your run

```
tail -f run.log # live bot log
curl http://localhost:4040/api/market/leaderboard # standings
curl http://localhost:4040/api/team/portfolio \
  -H 'Authorization: Bearer <jwt>' # your cash + positions
```

The web portal (<http://localhost:3030>) shows the same graphically — login, leaderboard, and your trade blotter.

8. Troubleshooting

Symptom	Cause / fix
INVALID_CREDENTIALS on login	Wrong email/password in config, or account not created yet
MARKET_NOT_ACTIVE:<state>	Event hasn't started / is frozen — wait for admin to start it
OUTSIDE_TRADING_WINDOW	Outside the admin-set daily algo window (IST)
RATE_LIMITED (HTTP 429)	Too many calls — slow down polling / pace orders
INSUFFICIENT_POSITION	Selling more than held — re-sync portfolio from server
Log looks empty	Run with <code>python3 -u</code> (stdout buffering)
Port already in use	Another instance is running — <code>lsof -ti :4040</code>
Sudden 401 mid-run	JWT expired — runner auto-relogins; ensure your client does too

9. Pre-event checklist

- ☐ Account created, login works, API key saved in config
- ☐ Sample bot connects with `--dry-run`
- ☐ Your strategy passes `test_algo.py` offline backtests
- ☐ Runner starts, prints symbols, and ticks without errors
- ☐ Rejection handling verified (force a bad order once and watch it recover)
- ☐ Strategy report PDF ready for submission (`POST /api/team/submission`)

Good luck!